4주차

클로드 코드 실전 활용

단계별 프롬프트로 할 일 관리 앱 만들기, 세션 관리, 프로젝트 개선

신구대학교 컴퓨터소프트웨어학과 · 앱프로그래밍응용 (2026-1)

오늘 배울 내용

이 수업의 3대 핵심 역량

역량	설명
바이브 코딩	AI 코딩 파트너를 활용, 자연어 프롬프트로 빠르게 개발
컴퓨팅 사고	문제 분해 → 패턴 인식 → 추상화 → 알고리즘적 해결
프로젝트 관리	요구사항 → 일정 계획 → 중간 보고 → 최종 발표

오늘의 학습 목표

1. **PRD를 단계별 프롬프트로 변환**하여 할 일 관리 앱을 구현할 수 있다.
2. **세션 재개와 컨텍스트 관리**로 장기 프로젝트를 효율적으로 진행할 수 있다.
3. **캡처, @ 기호, 설정 명령어**로 프로젝트를 개선하고 관리할 수 있다.

> 📖 *교재: 혼자 공부하는 바이브 코딩 with 클로드 코드 — Chapter 04 (p.128~171)*

지난 주 복습

3주차에서 배운 것

- **클로드 코드 설치** — Node.js, Git 설치 후 `npm install -g @anthropic-ai/claude-code`
- **손글씨 인식 프로그램** — MNIST 데이터셋으로 숫자 인식 앱 제작
- **CLAUDE.md 활용** — `/init`으로 프로젝트 설정 파일 생성, `#` 키로 규칙 추가

이번 주 핵심 질문

> 💬 *"PRD를 **단계별 프롬프트**로 나누면 왜 더 좋은 결과물이 나올까?"*

3주차까지는 한 번의 프롬프트로 프로그램을 만들었습니다. 이번 주부터는 **PRD → 단계별 프롬프트**로 기획과 구현을 나누고, 세션 관리와 프로젝트 관리까지 익힙니다!

이번 주의 컴퓨팅 사고

컴퓨팅 사고의 **4가지 사고력**이 이번 주에 어떻게 적용되는지 살펴봅시다.

사고력	의미	이번 주 적용
분해	큰 문제를 작은 단위로 나누기	PRD를 5단계 프롬프트로 분해
패턴 인식	반복되는 구조 찾기	Step별 "프롬프트→구현→확인" 사이클 반복
추상화	핵심만 뽑아 정리하기	PRD로 요구사항을 체계적으로 추상화
알고리즘	순서대로 절차 밟기	뼈대→기능→UI→고급→최적화 순서

💡 이번 주의 핵심은 **"분해"**! PRD라는 큰 문서를 5단계 프롬프트로 나누는 것이 바로 컴퓨팅 사고의 실전 적용입니다.

3대 역량, 이번 주 적용

역량	이번 주 활동
바이브 코딩	PRD 기반 단계별 프롬프트로 할 일 관리 앱 구현
컴퓨팅 사고	PRD 분해 + 세션/컨텍스트 관리의 체계적 접근
프로젝트 관리	세션 재개, 컨텍스트 압축, /export로 작업 기록 문서화

바이브 코딩 = PRD를 **단계별 프롬프트**로 나눠 구현 (도구)
컴퓨팅 사고 = 요구사항을 **체계적으로 분해** (사고)
프로젝트 관리 = 세션·컨텍스트·설정을 **효율적으로 관리** (계획)

단계별 프롬프트 전략이란?

단계별 프롬프트 전략이란 PRD를 기반으로 프로젝트를 여러 단계로 나누어 AI에게 구체적인 목표를 제시하는 방식입니다. (교재 p.145)

기획과 구현의 분리

기획 (클로드 웹)	구현 (클로드 코드)
PRD 작성	뼈대 → 기능 → UI → 고급 → 최적화

왜 단계별로 나눠야 할까?

- 한 번에 모든 기능을 요청하면 **품질이 떨어짐**
- 각 단계의 **결과를 확인한 뒤** 다음 단계 진행 가능
- 이전 단계의 결과물 위에 **점진적으로 기능 추가**

PRD 작성하기 (교재 p.129~130)

2장에서 익힌 **5W1H 프레임워크**로 클로드 웹에서 할 일 관리 앱의 PRD를 작성합니다.

```
할 일 관리 앱을 만들고 싶어. PRD를 작성해줘.

매일 10~20개 정도의 할 일을 관리하는 개인용 앱이야.
주요 기능은:
- 할 일 추가, 수정, 삭제
- 완료 체크 기능
- 카테고리 분류 (업무/개인/공부)
- 진행률 보기

브라우저에서 바로 실행되고, 새로고침해도 데이터가
사라지지 않았으면 좋겠어.
기술적으로 복잡하지 않게 순수 자바스크립트로 만들고 싶어.
```

PRD를 단계별 프롬프트로 변환하기 (교재 p.131)

생성한 PRD를 클로드 코드에서 실행할 수 있는 단계별 프롬프트로 변환합니다.

프롬프트: PRD → 단계별 프롬프트 변환 요청

```
이 PRD를 클로드 코드에서 사용할 단계별 프롬프트로 변환해 줘.
5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을
만들어 줘.
```

클로드가 제안하는 5단계

- **Step 1**: 기본 구조 및 핵심 기능 구현
- **Step 2**: 카테고리 기능 및 UI 개선
- **Step 3**: 진행률 대시보드 추가
- **Step 4**: 다크 모드 및 고급 기능
- **Step 5**: 최종 완성 및 최적화

단계별 프롬프트 실행하기 (교재 p.132~134)

클로드 코드 실행 준비

1. `VibeCoding` 폴더에 새 폴더 `Study-02` 생성
2. `Study-02` 폴더에서 터미널 열기 → `claude` 실행
3. `/init` 입력하여 프로젝트 초기 설정 (CLAUDE.md 자동 생성)

Step 1 프롬프트 입력

```
> Step 1: 기본 구조 및 핵심 기능 구현
할 일 관리 앱의 기본 구조를 만들어줘.
요구사항:
1. index.html, style.css, script.js 세 개의 파일로 구성
2. HTML 구조: 제목 "My Tasks", 할 일 입력창과 추가 버튼,
   할 일 목록 표시할 컨테이너
3. JavaScript 기능: 할 일 추가, 삭제, 체크박스로 완료 표시,
   localStorage에 데이터 저장
```

Step 1 실행 결과 (교재 p.135~136)

클로드 코드가 3개의 파일(`index.html`, `style.css`, `script.js`)을 생성합니다.

확인 방법

- `index.html` 파일을 더블 클릭하여 브라우저에서 실행
- 할 일 입력 → [추가] 버튼 클릭 → 목록에 추가되는지 확인
- 체크박스 클릭, 삭제 버튼 등 기능 테스트

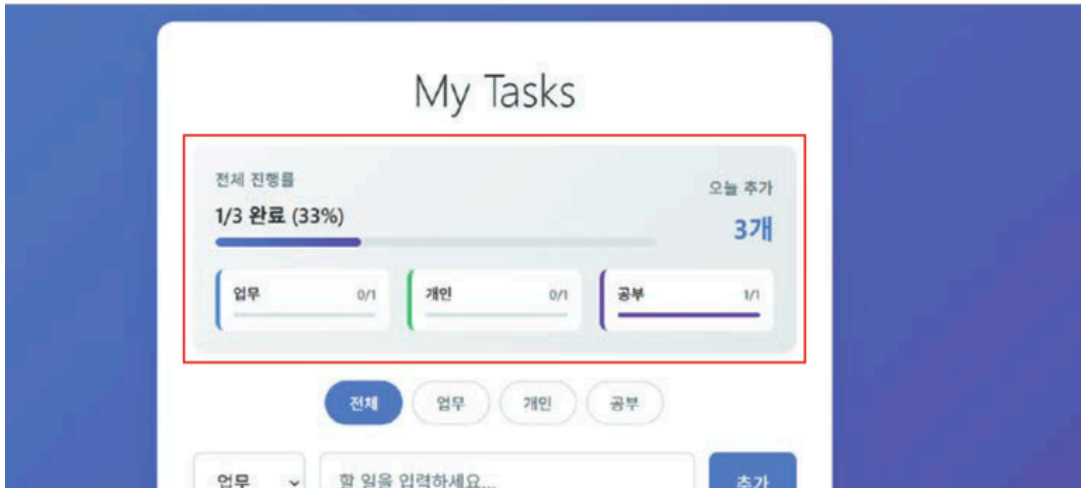
Step 2 프롬프트 입력

```
> Step 2: 카테고리 기능 및 UI 개선
기존 코드에 카테고리 기능을 추가하고 UI를 개선해줘.
요구사항:
1. 카테고리 기능: 업무(work), 개인(personal), 공부(study)
2. 카테고리별 색상 태그, 필터 버튼
3. 데이터 구조에 category 필드 추가
```

Step 3: 진행률 대시보드 추가 (교재 p.138~139)

- 전체 진행률 표시 ("5/10 완료 (50%)" 형식 + 프로그레스 바)
- 카테고리별 미니 진행률, 오늘 추가된 할 일 개수
- 인라인 수정 기능 (더블클릭으로 편집 모드)

관리 앱의 대시보드에 전체 진행률과 카테고리별 미니 진행률 등이 추가되었습니다.



Step 4~5: 다크 모드 & 최종 완성 (교재 p.140~141)

Step 4: 다크 모드 및 고급 기능

- 다크/라이트 모드 토글 스위치 + localStorage에 테마 저장
- "완료된 항목 모두 삭제" 버튼
- 키보드 단축키 (`Alt+N`: 새 할 일, `Alt+1,2,3,4`: 카테고리 필터)

Step 5: 최종 완성 및 최적화

- 데이터 내보내기/가져오기 (JSON 형식)
- 정렬 옵션 (생성일순, 카테고리순 등)
- 100개 이상 항목에서도 부드러운 성능

✏️ 셀프 체크 — 04-1 단계별 프롬프트

(교재 p.145~146)

문제 1. 클로드 코드에서 프로젝트 초기 설정을 위해 자주 쓰는 명령어에 대한 설명입니다. 빈칸에 알맞은 명령어들을 채워 보세요.

설명	명령어
프로젝트 구조를 분석하고 CLAUDE.md 파일을 자동 생성	
현재 세션의 토큰 사용량과 남은 컨텍스트 용량 확인	
사용할 AI 모델을 선택하거나 Default 옵션으로 변경	
사용 가능한 모든 명령어 목록과 설명을 확인	

✏️ 셀프 체크 — 04-1 (계속)

문제 2. 다음 설명을 보고 알맞은 용어를 쓰세요.

① 프로젝트 폴더 구조를 분석하여 자동으로 생성되는, 프로젝트의 컨텍스트와 규칙을 담은 마크다운 파일 → ( )

② 제품의 목적, 대상 사용자, 주요 기능, 기술 스펙 등을 체계적으로 정리한 제품 요구사항 문서 → ( )

③ 할 일 관리 앱에서 데이터를 파일로 저장하고 다시 불러올 때 사용하는 데이터 형식 → ( )

세션(Session)이란? (교재 p.147)

세션은 클로드 코드에서 대화를 시작해 명령을 주고받는 동안 유지되는 상태를 말합니다.

왜 세션 관리가 중요한가?

- 터미널을 닫으면 이전 대화가 사라진 것처럼 보임
- 실제로는 모든 대화 내용이 **프로젝트 폴더에 자동 저장**
- 며칠 후 이어서 작업해야 할 때 **세션 재개**가 필수

> 💡 장기 프로젝트를 진행할 때는 **세션과 컨텍스트 관리**에 특히 주의해야 합니다!

세션 재개하기 (교재 p.148~149)

방법 1: `claude --resume`

```
claude --resume
```

이전 대화 기록을 불러와서 마지막 상태 그대로 이어서 작업할 수 있습니다.

방법 2: `/resume` 명령어 (별칭: `/continue`)

`claude` 실행 후 `/resume`을 입력하면 이전 대화를 선택하여 재개합니다.

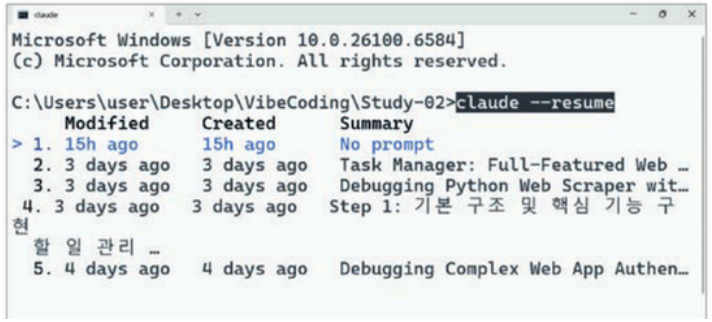
세션 재개하기 — 최근 대화 복구 (교재 p.149)

방법 3: `--continue` 또는 `-c` 옵션

```
claude --continue
```

가장 최근의 마지막 대화를 자동으로 복구합니다.

02 현재 프로젝트의 대화 목록을 보여줍니다. 화살표 키로 원하는 대화를 찾고, Enter 키를 눌러 재개합니다.



자동 승인 모드 (교재 p.150)

`Shift` + `Tab` 키를 누르면 **자동 승인 모드**로 전환됩니다.

자동 승인 모드란?

- 파일 수정 승인을 **매번 누르지 않아도** 자동으로 처리
- 반복적인 승인 요청을 생략하여 작업 속도 향상
- 다시 `Shift` + `Tab`을 누르면 일반 모드로 복귀

확인 표시

```
>> accept edits on (shift+tab to cycle)
```

> 💡 하단에 위 메시지가 표시되면 자동 승인 모드입니다. 편리하지만 코드 변경 사항을 **나중에 반드시 점검**하세요!

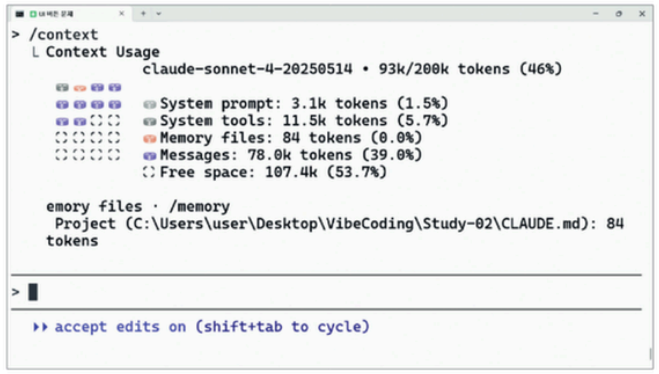
컨텍스트란? (교재 p.153~154)

- 현재 대화와 작업 정보를 저장 및 표시한 **메모리 공간**
- 대화가 길어지면 컨텍스트가 부족해져 자동 압축(auto-compact) 경고 발생

`/context` — 컨텍스트 상태 확인

```
> /context
```

현재 컨텍스트 사용량을 컬러 그리드로 시각화합니다.



note 토큰 한도는 내 구독 플랜에 따라 다릅니다.

컨텍스트 압축하기 (교재 p.155~156)

`/compact` — 대화 기록 요약

```
> /compact
```

대화 기록을 요약하여 컨텍스트 공간을 확보합니다.

> 압축 후에도 CLAUDE.md와 프로젝트 파일은 유지됩니다. 작업 단위가 끝날 때마다 **/compact**로 정리하는 습관을 들이세요!

✏️ 셀프 체크 — 04-2 작업 재개와 효율성

(교재 p.157)

문제 1. 다음 설명 중 틀린 것을 고르세요.

① /compact는 대화 내용을 요약하여 토큰을 절약한다.

② auto-compact는 컨텍스트가 임계값에 도달하면 자동으로 실행된다.

③ 압축 후에도 CLAUDE.md와 프로젝트 파일은 유지된다.

④ `Ctrl` + `R`은 /context 명령어의 단축키이다.

⑤ "Accept all" 모드는 `Shift` + `Tab`으로 전환할 수 있다.

✏️ 셀프 체크 — 04-2 (계속)

문제 2. 다음 상황에서 사용할 적절한 명령어나 옵션을 쓰세요.

상황	명령어 / 옵션
여러 대화 중 특정 대화를 선택해서 재개하고 싶을 때	
가장 최근 대화를 즉시 이어서 작업하고 싶을 때	
현재 컨텍스트 사용량을 시각적으로 확인하고 싶을 때	
대화가 길어져서 압축이 필요할 때	
파일 수정 승인을 매번 누르기 번거로울 때	

캡처 이미지로 UI 개선하기 (교재 p.159~163)

멀티모달 AI는 텍스트뿐만 아니라 이미지도 함께 이해하고 처리할 수 있습니다.

이미지 전달 방법

1. Windows **캡처 도구**(Snipping Tool) 실행
2. 할 일 관리 앱 화면을 **캡처**하여 이미지 저장
3. 저장한 이미지를 클로드 코드 명령 프롬프트 창에 **드래그 & 드롭**
4. 프롬프트 입력창에 `[Image #1]`이 표시됨

프롬프트 예시

```
[Image #1] 현재 디자인은 모바일에 최적화되어 있는 것 같아.
데스크톱 환경에서도 전체 화면에서 UI를 볼 수 있도록
디자인을 수정해 줘. 수정한 디자인은 'web_version'이라는
새로운 폴더를 생성한 뒤, 그곳에 저장해 줘.
```

@ 기호로 파일 분석하기 (교재 p.164~165)

@ 기호를 사용하면 파일이나 폴더를 즉시 참조하고 분석할 수 있습니다.

@ 기호로 파일 분석

```
@script.js 파일에서 다크 모드 관련 함수를 찾고 설명해 줘.
```

- 1,000줄이 넘는 코드에서 특정 함수를 빠르게 찾아 분석

> 💡 @ 기호를 사용하면 파일 전체를 훑어보지 않아도 **필요한 정보를 즉시 참조**할 수 있습니다!

@ 기호로 폴더 구조 확인하기 (교재 p.165~166)

@ 기호로 폴더 분석

```
@web_version 폴더에 있는 파일들을 분석해서 할 일 관리 앱이
데스크톱 환경에 최적화되었는지 확인해 줘.
```

- 폴더 내 모든 파일을 종합적으로 분석하여 평가
- 폴더 안의 파일 개수가 많아도 **종합적인 분석 결과** 제공

실습을 시작합니다!

지금부터 **할 일 관리 앱**을 단계별로 만들고, **세션 관리**와 **프로젝트 관리**를 실습합니다.

실습 흐름

```
① PRD 작성        →  ② 단계별 프롬프트 변환  →  ③ Step 1~3 구현  →
④ 세션 재개 실습  →  ⑤ 캡처로 UI 개선       →  ⑥ @ 기호 · 명령어 실습
```

준비물

- 인터넷 연결된 PC (Windows)
- 명령 프롬프트 (cmd) 또는 터미널
- Claude 계정 (https://claude.ai)
- `VibeCoding/Study-02` 폴더 생성

실습 ① PRD 작성

클로드 웹(claude.ai)에서 아래 프롬프트를 입력하여 할 일 관리 앱의 PRD를 요청합니다.

```
할 일 관리 앱을 만들고 싶어. PRD를 작성해줘.

매일 10~20개 정도의 할 일을 관리하는 개인용 앱이야.
주요 기능은:
- 할 일 추가, 수정, 삭제
- 완료 체크 기능
- 카테고리 분류 (업무/개인/공부)
- 진행률 보기

브라우저에서 바로 실행되고, 새로고침해도 데이터가
사라지지 않았으면 좋겠어.
기술적으로 복잡하지 않게 순수 자바스크립트로 만들고 싶어.
```

실습 ① PRD → 단계별 프롬프트 변환

2단계: PRD를 단계별 프롬프트로 변환

```
이 PRD를 클로드 코드에서 사용할 단계별 프롬프트로 변환해 줘.
5개의 핵심 단계로 정리해서 각 단계마다 명확한 지시 사항을
만들어 줘.
```

클로드가 제안한 5단계를 메모장에 저장

변환된 단계별 프롬프트를 **메모장이나 다른 텍스트 편집기**에 저장해두세요.

[확인] 체크리스트

- [ ] 클로드 웹에서 PRD가 정상적으로 생성되었는가?
- [ ] PRD에 핵심 기능, 기술 스택, UI 요구사항이 포함되었는가?
- [ ] 5단계 프롬프트가 메모장에 저장되었는가?

실습 ② 클로드 코드에서 단계별 구현

프로젝트 폴더 준비 및 클로드 코드 실행

```
mkdir Study-02
cd Study-02
claude
```

/init으로 프로젝트 초기 설정

```
/init
```

Step 1 프롬프트 입력 (복사 → 붙여넣기)

저장해 둔 단계별 프롬프트의 **Step 1을 복사**하여 클로드 코드에 붙여 넣습니다.

> 💡 터미널에서 여러 행의 프롬프트를 직접 입력할 때는 `↵` + `Enter` 또는 `Ctrl` + `J` 키를 눌러 줄바꿈할 수 있습니다.

실습 ② Step 1~3 구현 및 확인

Step별 구현 → 확인 → 다음 단계 반복

Step	구현 내용	확인 방법
Step 1	기본 구조 + CRUD 기능	`index.html` 브라우저 실행
Step 2	카테고리 + UI 개선	필터 버튼·색상 태그 동작 확인
Step 3	진행률 대시보드	프로그레스 바·인라인 수정 확인

[확인] 체크리스트

- [ ] 할 일 추가/수정/삭제가 정상 동작하는가?
- [ ] 카테고리별 필터링이 작동하는가?
- [ ] 진행률 대시보드가 정확히 표시되는가?
- [ ] 새로고침 후에도 데이터가 유지되는가? (localStorage)

실습 ③ 세션 재개 실습

클로드 코드 종료 후 다시 시작

1. `Ctrl` + `C` 두 번으로 클로드 코드 **종료**
2. 터미널을 **닫았다가** 다시 열기
3. `Study-02` 폴더에서 세션 재개

```
claude --resume
```

이전 대화 목록에서 선택

- 화살표 키로 원하는 대화를 찾아 `Enter` 키로 재개
- 이전 작업 상태가 복원되어 이어서 작업 가능

Step 4~5 프롬프트 입력

세션이 재개되면 **Step 4(다크 모드)**, **Step 5(최종 완성)** 순서로 진행합니다.

실습 ③ 세션 재개 확인

[확인] 체크리스트

- [ ] `claude --resume`으로 이전 대화 목록이 표시되는가?
- [ ] 세션 재개 후 이전 작업 맥락이 유지되는가?
- [ ] Step 4 — 다크 모드 토글이 동작하는가?
- [ ] Step 5 — 내보내기(JSON)와 가져오기가 동작하는가?

> **컴퓨팅 사고 연결**: 세션 재개는 **"알고리즘적 사고"**의 실전 적용입니다. 작업 순서를 체계적으로 기록하고, 중단된 지점부터 정확히 재개하는 것이죠.

실습 ④ 캡처로 UI 개선 & @ 기호 코드 점검

캡처 이미지로 데스크톱 UI 개선

1. `Windows` + `Shift` + `S` 로 앱 화면 캡처
2. `Study-02` 폴더에 `화면캡처.png` 로 저장
3. 이미지를 클로드 코드 프롬프트 창에 **드래그 & 드롭**

@ 기호로 코드 분석

```
@script.js 파일에서 다크 모드 관련 함수를 찾고 설명해 줘.
```

```
@web_version 폴더에 있는 파일들을 분석해서 할 일 관리 앱이
데스크톱 환경에 최적화되었는지 확인해 줘.
```

실습 ⑤ 프로젝트 관리 명령어 실습

/config — 설정 확인 및 변경 (교재 p.167)

```
> /config
```

설정 항목	기본값	설명
Auto-compact	TRUE	컨텍스트 자동 압축
Theme	Light mode	터미널 테마 설정
Model	Default	사용할 AI 모델 선택

> 💡 *최신 버전에서는 `/settings` 도 같은 기능으로 사용 가능합니다.*

실습 ⑤ /doctor 시스템 진단 (교재 p.168)

/doctor — 시스템 진단

```
> /doctor
```

클로드 코드의 설치 상태, 버전, 업데이트 가능 여부 등을 점검합니다.

점검 항목

항목	설명
설치 방식	npm global 또는 다른 방식으로 설치되었는지
버전 정보	현재 설치된 버전과 최신 버전 비교
자동 업데이트	자동 업데이트 설정 상태

실습 ⑤ /export로 대화 내보내기 (교재 p.168~169)

/export 명령어 실행

```
> /export
```

내보내기 옵션 선택

옵션	설명
1. Copy to clipboard	전체 대화를 클립보드로 복사
2. Save to file	대화 내용을 파일로 저장

[확인] 체크리스트

- [ ] /config로 현재 설정이 표시되는가?
- [ ] /doctor 진단 결과에 문제가 없는가?
- [ ] /export로 대화 기록이 파일로 저장되었는가?

실습 중 자주 발생하는 문제

문제	원인	해결 방법
`claude --resume`에 목록이 없음	다른 폴더에서 실행	`Study-02` 폴더로 이동 후 재실행
컨텍스트 부족 경고	대화가 너무 길어짐	`/compact` 명령어로 압축
Step 프롬프트 붙여넣기 오류	줄바꿈 호환 문제	`↵` + `Enter`로 줄바꿈 입력
이미지 드래그 & 드롭 안 됨	터미널 호환 문제	이미지 파일 경로를 직접 입력
/config 설정 변경 안 됨	방향키 미사용	`↑`, `↓` 키로 항목 선택 후 `Enter` / `Tab` / `Space`로 변경

핵심 키워드 — 이론 (교재 p.145, 157)

- **단계별 프롬프트 전략** — PRD를 기반으로 프로젝트를 여러 단계로 나누어 구체적인 목표를 제시하는 프롬프트 설계 방식
- **PRD** — Product Requirements Document, 제품의 목적·기능·기술 스펙을 체계적으로 정리한 문서
- **세션(Session)** — 클로드 코드에서 대화를 시작해 명령을 주고받는 동안 유지되는 상태
- **컨텍스트(Context)** — 현재 대화와 작업 정보를 저장 및 표시한 메모리 공간
- **멀티모달 AI** — 텍스트뿐만 아니라 이미지, 음성 등 서로 다른 형태의 데이터를 함께 인식할 수 있는 AI

핵심 키워드 — 실습 ① (교재 p.157)

- **--resume** — 이전 대화 목록에서 원하는 세션을 선택해 복원하는 명령어
- **--continue (-c)** — 가장 최근 대화를 자동으로 복구하는 플래그
- **자동 승인 모드** — `Shift` + `Tab` 으로 전환, 반복적인 권한 승인을 생략하는 모드
- **/compact** — 대화 기록을 요약해 컨텍스트 공간을 확보하는 명령어

핵심 키워드 — 실습 ② (교재 p.171)

- **@ 기호** — 파일 및 폴더를 즉시 참조하고 분석할 수 있는 기호
- **/config** — 프로젝트 환경 설정을 확인하고 관리하는 명령어
- **/doctor** — 시스템 상태를 진단하여 버전, 설치 방식, 업데이트 등을 점검하는 명령어
- **/export** — 작업 기록을 클립보드나 파일에 저장하고 문서화하는 명령어

✏️ 셀프 체크 — 04-3 프로젝트 개선과 작업 관리

(교재 p.171)

문제 1. 다음 설명 중 틀린 것을 고르세요.

① 스크린 캡처한 이미지는 드래그앤드롭으로 전달할 수 있다.
② @ 기호를 사용하면 파일 검색 시간 없이 즉시 프롬프트에 포함된다.
③ /config는 클로드 코드의 전역 설정만 변경할 수 있다.
④ /doctor는 자동 업데이트 설정 상태를 확인할 수 있다.
⑤ /export로 저장한 파일은 CLAUDE.md 작성 시 참고할 수 있다.

✏️ 셀프 체크 — 04-3 (계속)

문제 2. 다음 상황에서 사용할 적절한 명령어나 기능을 쓰세요.

상황	명령
web_version 폴더 구조를 확인하고 싶을 때	
script.js 파일에서 특정 함수를 찾고 싶을 때	
클로드 코드 설치 상태를 점검하고 싶을 때	
작업한 대화 내용을 파일로 저장하고 싶을 때	
다크 모드로 변경하고 싶을 때	
자동 압축 기능을 끄고 싶을 때	

오늘의 마무리 체크리스트

수업이 끝나기 전에 아래 항목을 확인해 보세요!

- [ ] 클로드 웹에서 **PRD를 작성**하고 **단계별 프롬프트로 변환** 완료
- [ ] 클로드 코드에서 **Step 1~5**를 순서대로 실행하여 할 일 관리 앱 완성
- [ ] `claude --resume`으로 **세션 재개**를 직접 실습
- [ ] `/compact`로 **컨텍스트 압축**을 실행해 보기
- [ ] 캡처 이미지 전달 또는 **@ 기호**로 코드 분석 실습
- [ ] `/config`, `/doctor`, `/export` 명령어 한 번씩 실행
- [ ] "내가 오늘 배운 것" 2줄 메모

> ✅ 할 일 관리 앱 실행 화면 캡처 + /export로 저장한 대화 기록을 교수님께 제출하면 오늘 수업 완료!

📌 최신 업데이트 — 알아두면 유용한 추가 명령어

교재에는 없지만, **최신 Claude Code**에서 추가된 유용한 명령어들입니다.

명령어	설명
`/clear`	대화 기록을 완전히 지우고 컨텍스트를 확보 (별칭: `/reset`, `/new`)
`/cost`	토큰 사용량 통계를 확인
`/diff`	커밋되지 않은 변경 사항을 인터랙티브 뷰어로 확인
`/memory`	CLAUDE.md 메모리 파일을 편집하고 자동 메모리 관리
`/effort`	AI 모델의 응답 노력 수준 조절 (low/medium/high)
`/rename`	현재 세션에 이름을 지정하여 나중에 쉽게 재개

💡 이 명령어들은 최신 버전에서만 제공될 수 있습니다. `/help`로 사용 가능한 명령어 목록을 확인하세요!

과제 안내

이번 주 과제

오늘 만든 할 일 관리 앱을 **데스크톱 버전**(`web_version`)으로 확장하고, 추가 기능 1가지 이상을 직접 프롬프트를 작성하여 구현하세요.

과제 요구사항

- 캡처 이미지를 활용한 **데스크톱 UI 개선** 완료
- 본인이 직접 작성한 **추가 기능 프롬프트** 1개 이상
- `/export`로 **대화 기록** 저장 후 제출
- 실행 결과 **스크린샷** 제출

다음 주 예고

> ***5주차**: 게임 제작으로 배우는 체계적인 개발과 관리 — 할루시네이션 해결, 커스텀 명령어, 자동화 워크플로*

오늘 배운 "단계별 프롬프트"와 "세션·컨텍스트 관리"는 앞으로 **모든 프로젝트에서 반복적으로 사용**하는 핵심 기법입니다!